

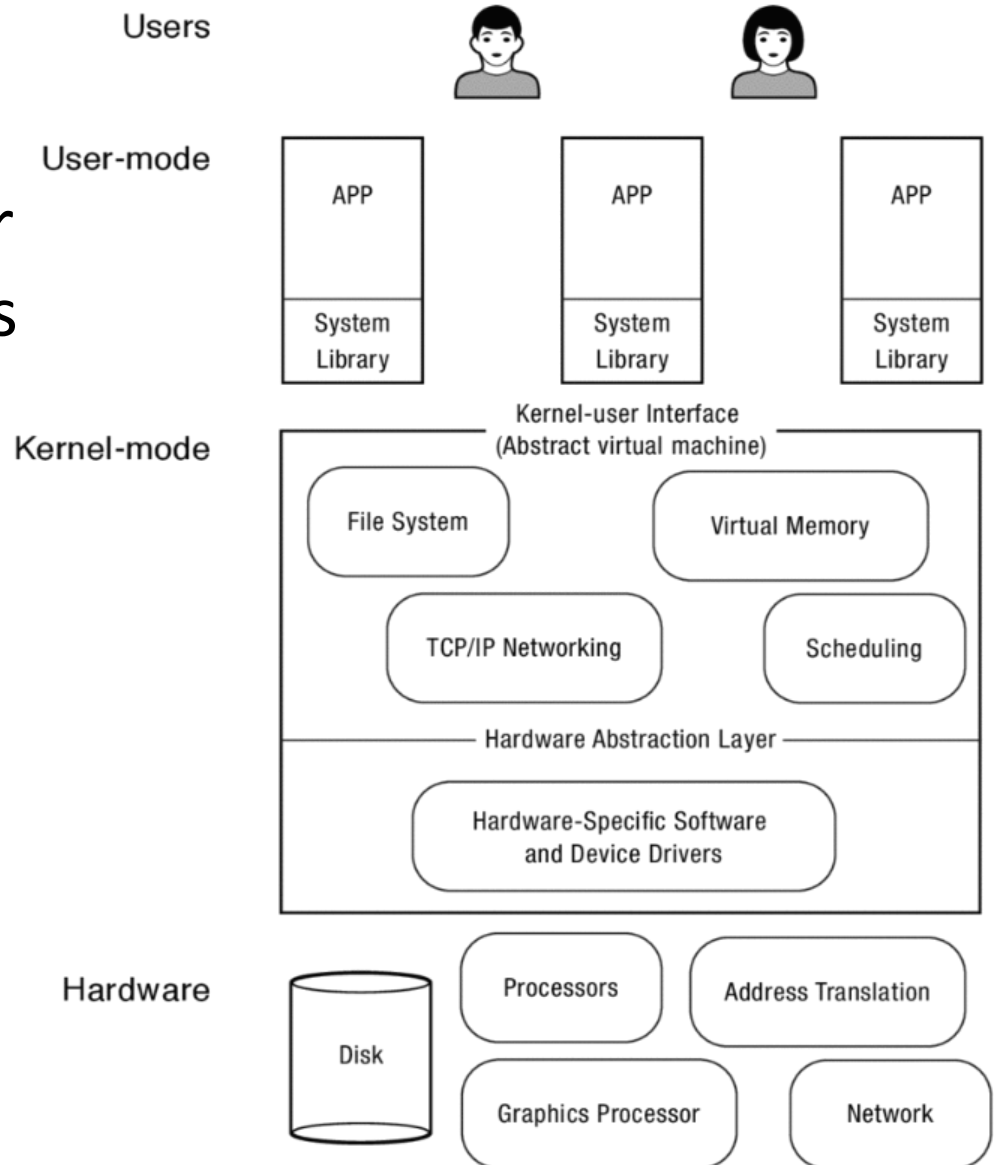
Operating Systems: Principles and Practice

Main Points

- Operating system definition
 - Software to manage a computer's resources for its users and applications
- OS challenges
 - Reliability, security, responsiveness, portability, ...
- OS history
 - Where we are?

What is an operating system?

- Software to manage a computer's resources for its users and applications
- Core functionalities:
 - Scheduling of resources
 - Virtual Memory
 - Inter-Process Communication (IPC) and synchronization mechanisms



Operating System Roles

- Referee:
 - Resource allocation among users, applications
 - Isolation of different users, applications from each other
 - Communication between users, applications
- Illusionist
 - Each application appears to have the entire machine to itself
 - Infinite number of processors, (near) infinite amount of memory, reliable storage, reliable network transport
- Glue
 - Provide common, standard services to applications
 - Simplifies application development
 - Libraries, user interface widgets, ...

Example: File Systems

- Referee
 - Prevent users from accessing each other's files without permission
 - Even after a file is deleting and its space re-used
- Illusionist
 - Files can grow (nearly) arbitrarily large
 - Files persist even when the machine crashes in the middle of a save
- Glue
 - Named directories, printf, ...

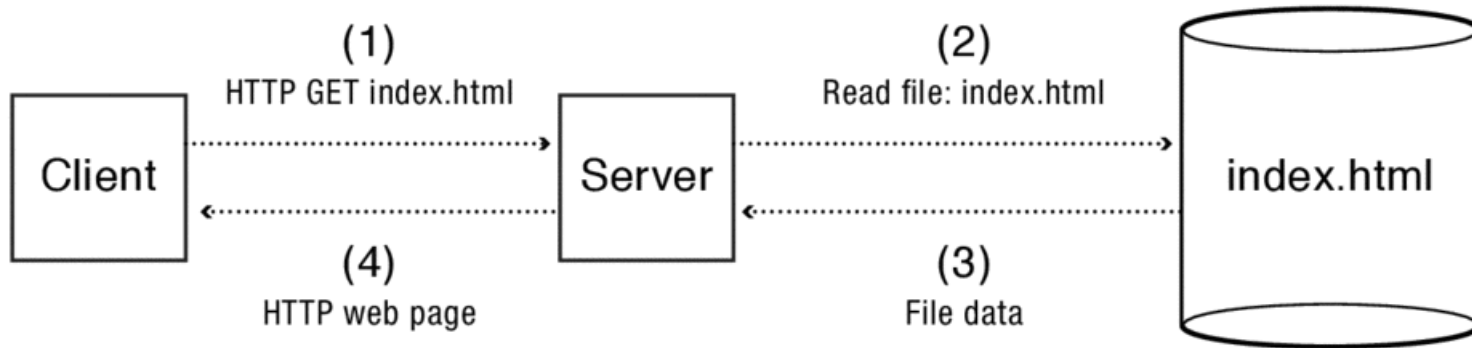
Operating system design patterns

- Cloud computing
 - Referee: how to allocate resources between competing applications in the cloud?
 - Illusionist: computing resources in a cloud evolve continuously, how to isolate applications from this evolution?
 - Glue: how to provide common, standardized access to the cloud services?
- Web services
 - Referee: ensure responsiveness when multiple tabs are opened at the same time
 - Illusionist: web services are geographically distributed for fault tolerance. Mask server failures to the users.
 - Glue: how does a browser achieve portable execution of scripts across different OS and HW platforms?

Operating system design patterns

- Multi-user database systems
 - Referee: how to enforce data access and privacy to different users ?
 - Illusionist: how to mask failures so that data remains consistent and available to users?
 - Glue: what common services to programs development?
- Internet
 - Referee: guarantee differentiated services to users and protect against DoS, spam, phishing etc...
 - Illusionist: internet appears as a unique, world-wide network but it is not!
 - Glue: internet protocols make applications independent of the underlying network architecture

Example: web service



- It defines a simple behavior but ...
- How does the server manage many simultaneous client requests?
- How do we keep the client safe from spyware embedded in scripts on a web site?
- How do make updates to the web site so that clients always see a consistent view?

Example: web service

However:

- Multiple users issue requests at the same time
 - These must be managed simultaneously
- Many requests involve data and computations
 - Think about search engines, a request may involve deep computations over large clusters of machines
- The server uses caches to speed up
 - Cache is shared among users, need for synchronized access mechanisms
- Servers send to clients scripts for pages customization
 - How does the client can protect itself from the execution of third party code that may embed viruses/spyware?

Example: web service

However:

- Web sites need to be updated
 - How to manage consistency with concurrent read requests?
- Client and server may run at different speeds
 - Need for speed decoupling
- Hardware supporting the web site may be updated
 - How to take advantage of this without rewriting the web server code?

Question

- What (hardware, software) do you need to be able to run an untrustworthy application?

OS Challenges

- Reliability
 - Does the system do what it was designed to do?
 - Availability
 - What portion of the time is the system working?
 - Mean Time To Failure (MTTF), Mean Time to Repair
- Security
 - Can the system be compromised by an attacker?
 - Privacy
 - Data is accessible only to authorized users
- Both require very careful design and code

OS Challenges

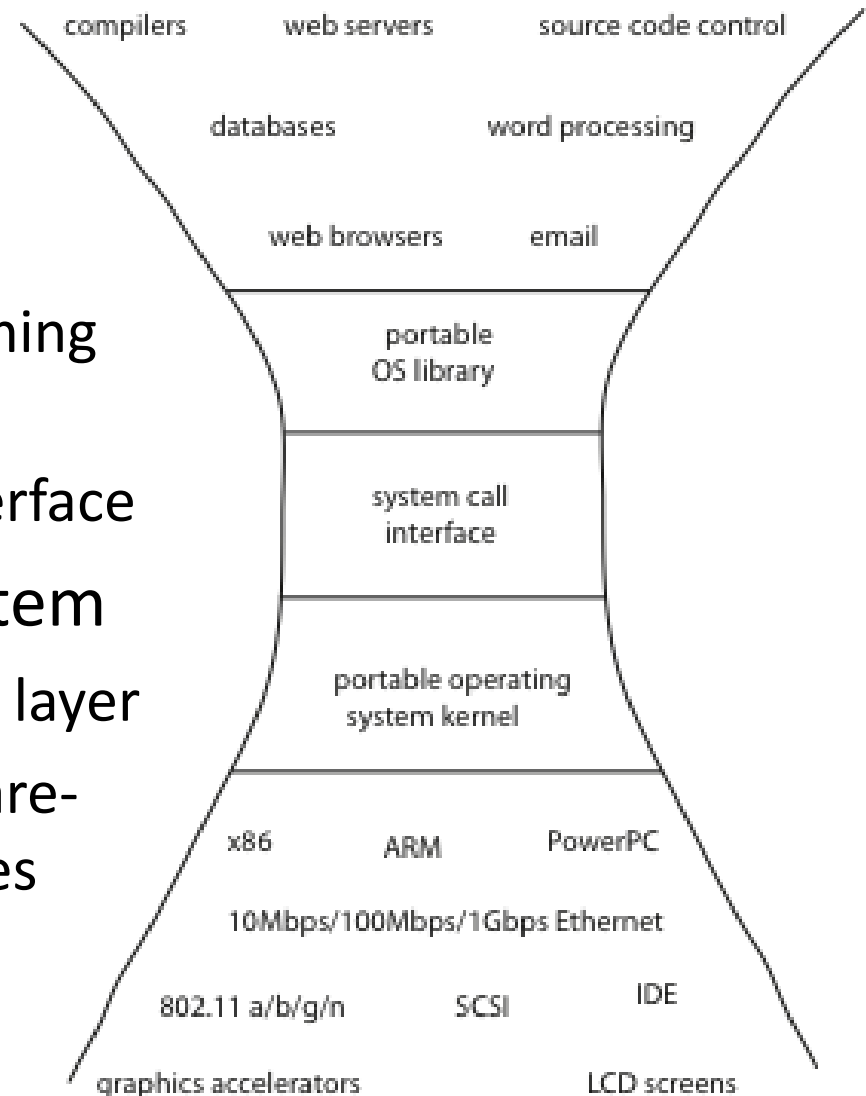
- Portability

- For programs:

- Application programming interface (API)
 - Abstract machine interface

- For the operating system

- Hardware abstraction layer
 - Most OS have hardware-specific kernel routines



OS Challenges

- Performance
 - Latency/response time
 - How long does an operation take to complete?
 - Throughput
 - How many operations can be done per unit of time?
 - Overhead
 - How much extra work is done by the OS?
 - Fairness
 - How equal is the performance received by different users?
 - Predictability
 - How consistent is the performance over time?

OS Structure

- Many dependencies among modules
 - Many parts of the OS depends on synchronization primitives
 - The Virtual Memory system depends on low-level HW support for address translation
 - Both the File System and the Virtual Memory share blocks of physical memory
 - The File System can depend on the network protocol stack
 - ...

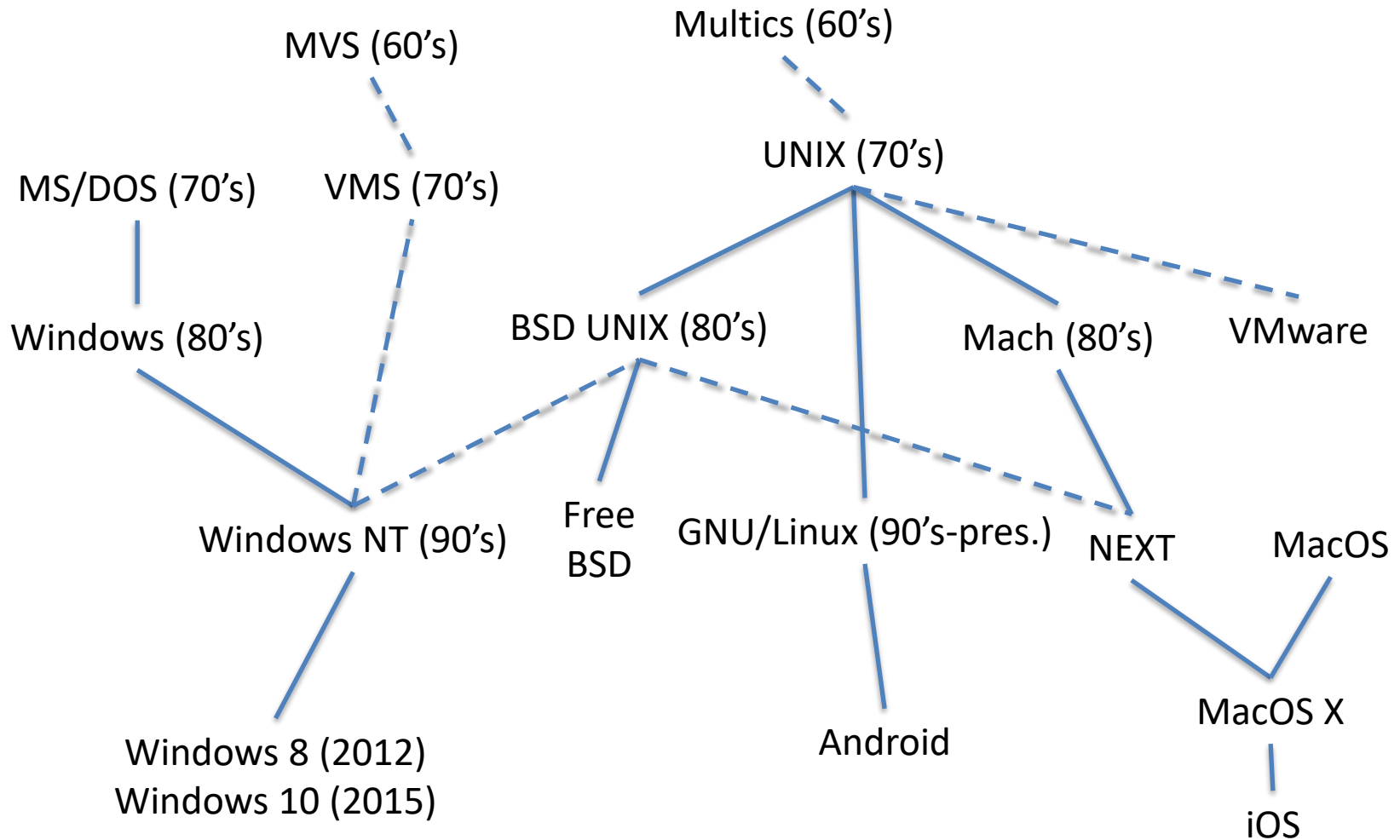
OS Structure

- Tradeoff in the OS design:
 - **Monolithic kernel**
 - Most of the OS functionalities run inside the kernel
 - To improve portability and flexibility monolithic kernels use HAL and dynamically loaded device drivers
 - **Microkernel**
 - In the kernel only core mechanisms, most of the OS functionalities run as user-level servers
 - More reliable due to the higher isolation; easier to debug, maintain and extend
 - **Hybrid model**
 - Combines the best practices of the two previous approaches

OS Adoption

- Adoption is beyond control of an OS
 - Wide availability of applications
 - Wide availability of HW supporting it
- Network effect
 - App stores
 - Example: Android model vs iPhone model
- Proprietary vs open systems
 - Not a clear winner

OS History

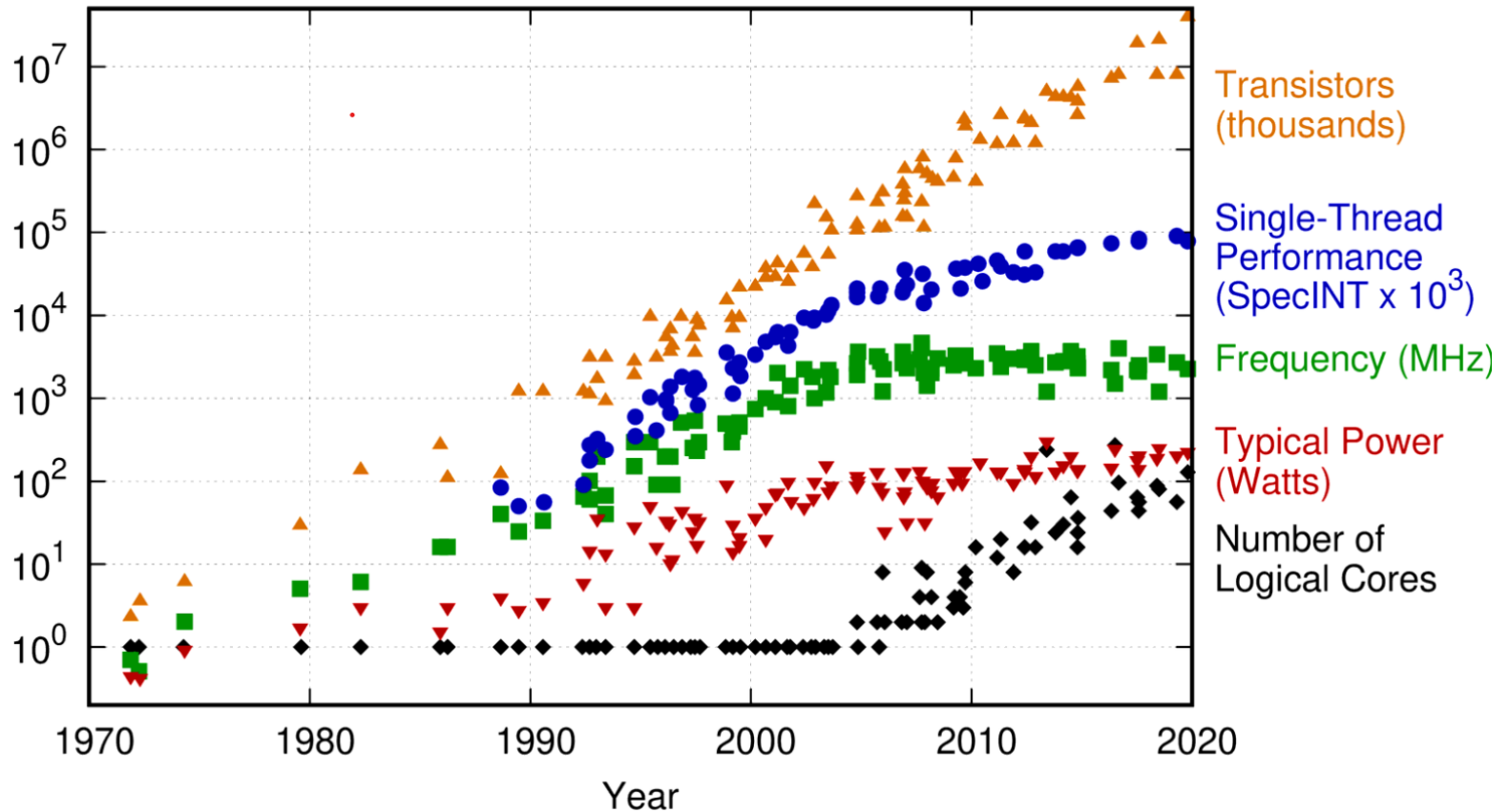


Computer Performance Over Time

	1981	1997	2014	Factor
Processor Speed MIPS	1	200	2500	2.5K
CPUs per computer	1	1	10+	10+
Processor \$ /MIPS	\$100 K	\$25	\$0.20	500 K
DRAM capacity (MiB/\$)	0.002	2	1K	500 K
Disk capacity (GiB) / \$	0.003	7	25 K	10 M
Home Internet	300 bps	256 Kbps	20 Mbps	100 K
LAN Network	10 Mbps shared	100 Mbps switched	10 Gbps switched	1000+
Users per machine	100:1	1:1	1:several	100+

Microprocessor Trend

48 Years of Microprocessor Trend Data



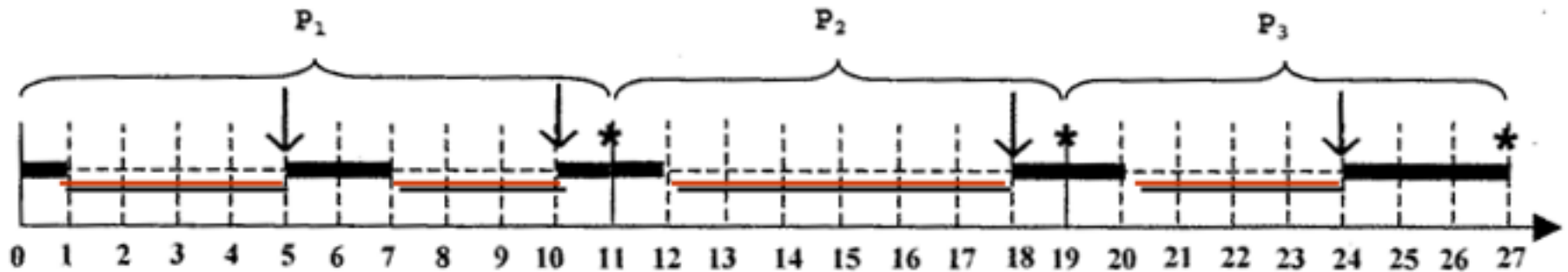
Source: <https://github.com/karlrupp/microprocessor-trend-data>

Early Operating Systems: Computers Very Expensive

- One application at a time
 - Had complete control of hardware
 - OS was runtime library
 - Users would stand in line to use the computer
- Batch systems
 - Keep CPU busy by having a queue of jobs
 - OS would load next job while current one runs
 - Users would submit jobs, and wait, and wait, ...

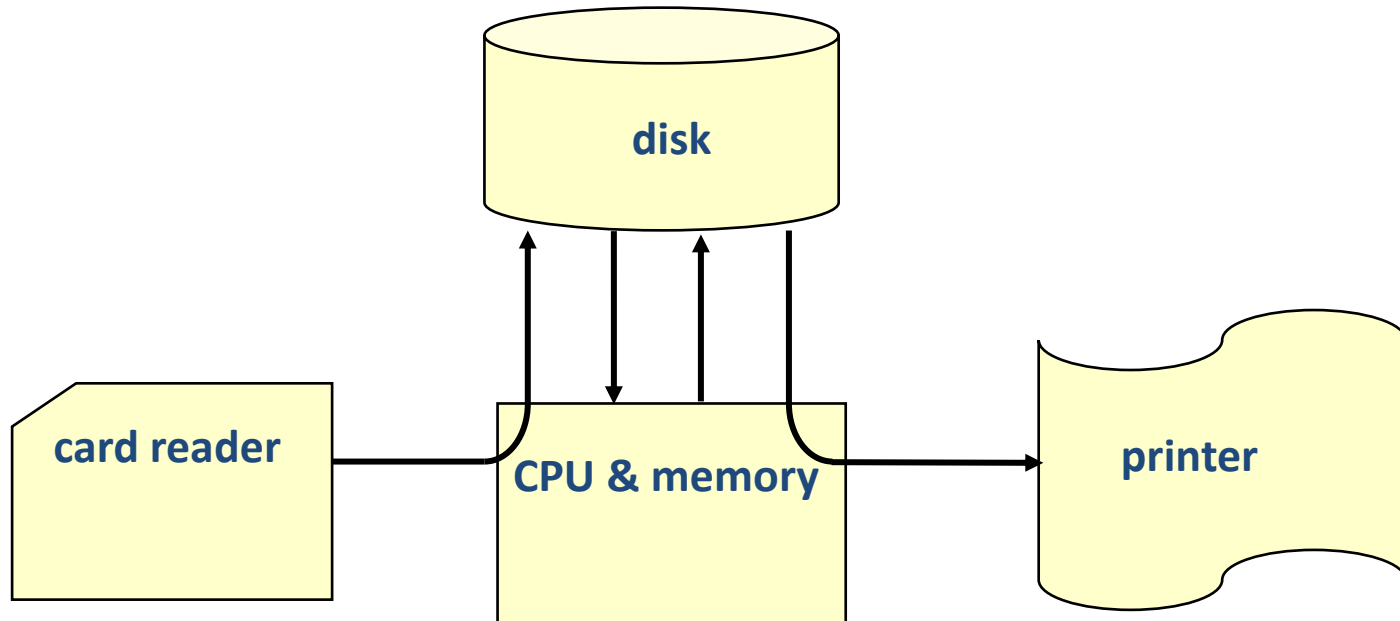
Single task systems

- Sequential execution
 - Black line: execution on the processor
 - Red line: I/O operation – the program stops & waits for the result of the operation



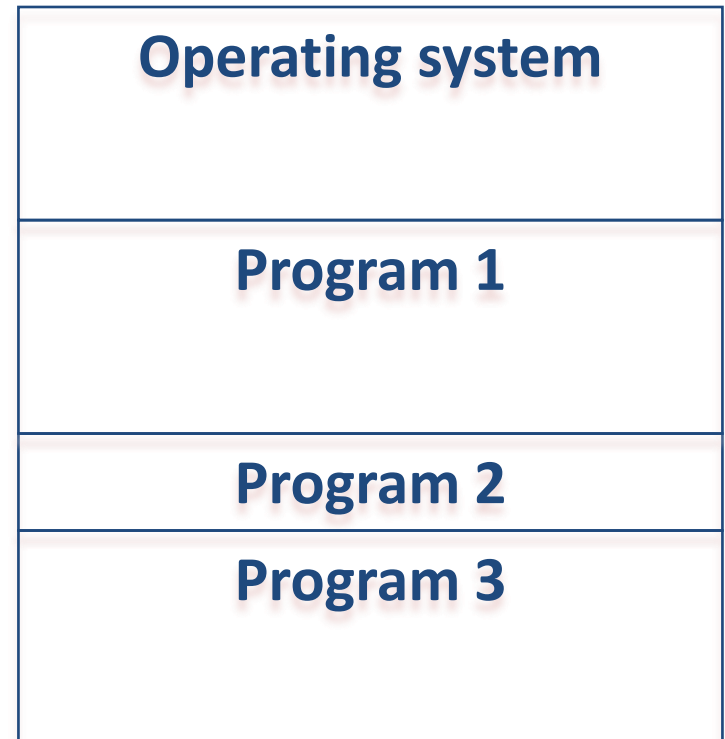
Early batch systems

- SPOOL: Simultaneous Peripheral Operation On-Line

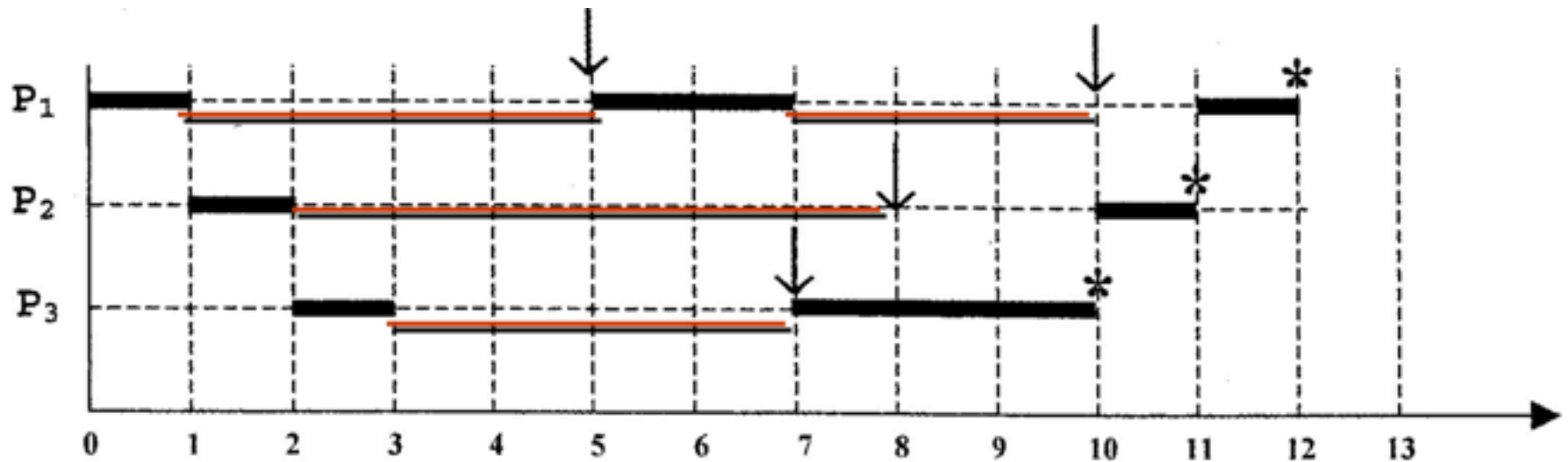
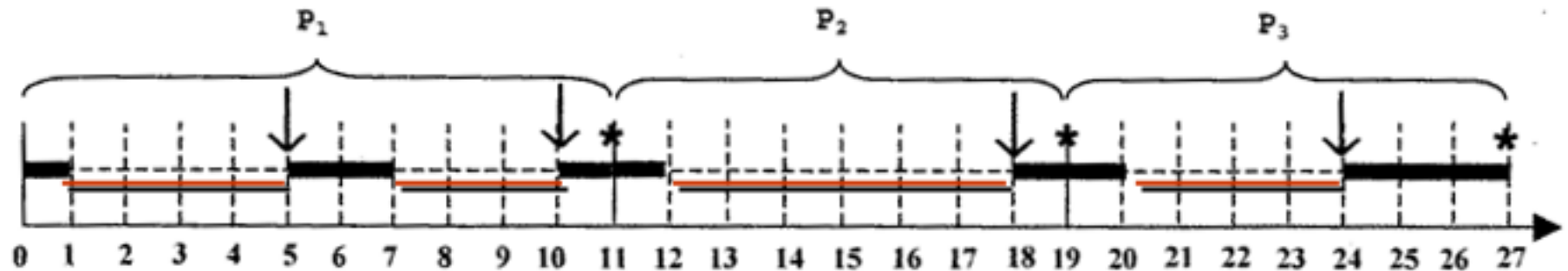


Multi-programmed batch systems

- multi-user system: several programs loaded in memory at the same time
- Spool optimization
- Resource optimization (processor, memory, devices)
 - Response time not important



Multi-tasking vs single-task

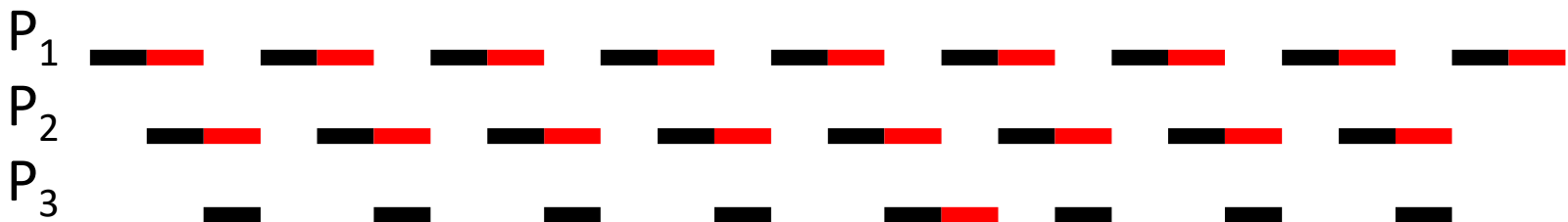
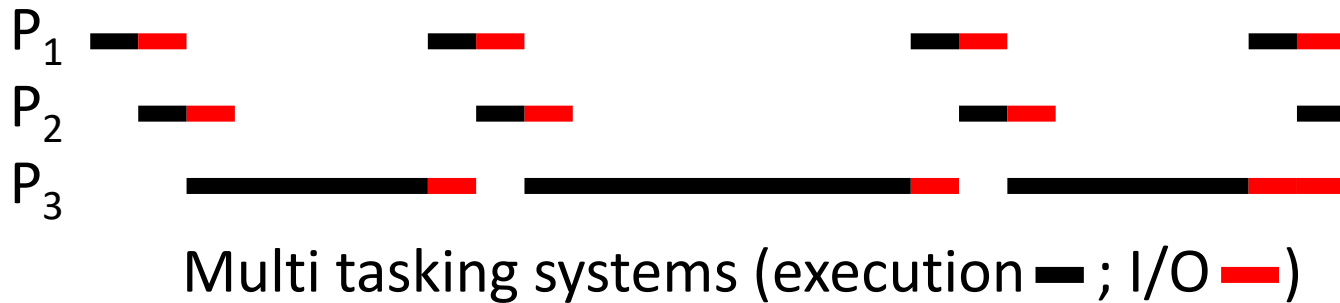


Time-Sharing Operating Systems: Computers and People Expensive

- Multiple users on computer at same time
 - Multiprogramming: run multiple programs at same time
 - Interactive performance: try to complete everyone's tasks quickly
 - As computers became cheaper, more important to optimize for user time, not computer time

Time-Sharing Operating Systems

- time sharing v.s. multitasking



Time sharing systems (execution — ; I/O —)

Today's Operating Systems: Computers Cheap

- Smartphones
- Embedded systems
- Laptops
- Tablets
- Virtual machines
- Data center servers

Tomorrow's Operating Systems

- Giant-scale data centers
- Increasing numbers of processors per computer
- Increasing numbers of computers per user
- Very large-scale storage (cloud storage)

New scenarios:

- Edge Computing
- Internet of Things
- Green Computing

Bonus Thought Questions

- How should an operating system allocate processing time between competing uses?
 - Give the CPU to the first to arrive?
 - To the one that needs the least resources to complete? To the one that needs the most resources?
- What if you need to allocate memory?